

Track 15 — Was eine Sperre bewirkt, und wer wen bewertet

Datum: 2026-08-28 **Ausgangsstand:** 022556e (Track 14), 1217 Tests **Endstand:** 1236 Tests (19 neu), Typecheck grün

Warum diese Strecke

Track 14 hat drei Punkte offen benannt, zwei davon fallen hierher: `is_active / is_verified` auf der Mietstrecke und ein `passwordMustChange`, das nie gesetzt wird. Dazu die Frage, die noch keiner der bisherigen Tracks gestellt hat: kann Anbieter A an die Daten von Anbieter B?

Die Antwort auf die letzte Frage ist erfreulich kurz und steht deshalb ganz unten. Der Rest dieses Berichts handelt von einem Hebel, der aussah, als würde er etwas bewirken.

Befunde

1 — P0: Der gesperrte Salon nahm weiter Geld ein

`salons.is_active` ist der **einzige** Hebel, mit dem die Plattform einen Anbieter anhalten kann. `/admin/anbieter` schreibt ihn an zwei Stellen:

```
// PATCH /api/admin, action 'salon-status'
} else if (data.status === 'suspended') {
  updates.is_active = false
}
// action 'salon-toggle-active' → „🔴 Offline setzen"
```

Das ist die Reaktion auf Betrug, auf eine Beschwerde, auf eine fehlende Gewerbeanmeldung. Der Knopf heißt „Sperren“.

Gesperrt hat er die Schaufenster. Startseite, Suche, Stadt- und Kategorieseiten, Sitemap und `/listings/[slug]` filtern alle mit `.eq('is_active', true)` — der Salon verschwand also aus den Listen. Jede Strecke, auf der Geld oder eine Verpflichtung entsteht, hat `salons` dagegen überhaupt nicht angefasst:

Strecke	Zustand vorher
<code>createBooking</code>	lädt <code>services</code> , den Salon nie — Termin wurde angenommen
<code>GET /api/availability</code>	bot weiter das volle Slot-Raster an
<code>GET /api/rental-listings</code>	die einzige öffentliche Liste ohne den Filter
<code>GET /api/rental-equipment/[id]</code>	Buchungs-/Anfrageformular ging normal auf
<code>POST /api/rental-bookings</code>	echte Stripe-Checkout-Session, Geld wurde eingezogen
<code>POST /api/rental-requests</code>	Anfrage zugestellt, inkl. E-Mail an den Vermieter

Die teuerste Zeile ist die vorletzte. Der Ablauf einer Miete beim gesperrten Anbieter war vollständig:

1. `/api/rental-bookings` legt die Buchung an und erzeugt die Stripe-Session — der Mieter zahlt. 2. Der Webhook (`rental_payment`) setzt `paid / confirmed` und schreibt eine `platform_transactions` - Zeile mit `provider_user_id = salons.owner_id`. 3. `cron/rental-payouts` überweist `provider_share_cents` beim Mietbeginn an den Connect-Account genau dieses Anbieters.

Kein Glied dieser Kette fragt nach `is_active`. Ein wegen Betrugs gesperrter Anbieter war also nur schwerer zu *finden* — angehalten war er nicht. Jeder Direktlink (`/salon/<slug>`, `/inserat/<id>`) und jeder API-Aufruf lief unverändert durch, und die Mietsuche zeigte seine Inserate ohnehin weiter.

Jetzt: ein gemeinsamer Riegel in `src/lib/salon-status.ts`, angewendet auf alle sechs Stellen. Die Geldstrecken sind *fail closed* — ein Lesefehler beim Prüfen des Salons sperrt, statt durchzulassen; dieselbe Linie wie in `getServerSession`.

Zwei Entscheidungen, die bewusst NICHT getroffen wurden:

- **`is_active: null` sperrt nicht.** Der Wert lässt sich mit dem ANON-Key nicht auslesen (`salons` antwortet für `anon` mit 42501 aus `is_admin_or_super`). Aus „ich kenne den Default nicht“ eine Sperre zu machen hieße, laufende Buchungen auf eine Vermutung hin abzuschalten. Der Admin-Hebel schreibt immer einen echten Boolean.
- **`is_verified` sperrt nicht.** Ein frisch registrierter Salon ist `is_verified: false` **und** `is_active: true` — das Admin-Dashboard nennt diesen Zustand „pending“ und zeigt ihn als arbeitsfähig. Die Prüfung daran zu hängen würde jeden noch nicht freigeschalteten Anbieter sofort vom Markt nehmen. **Damit lautet die Antwort auf „kann ein nicht-verifizierter Salon Buchungen empfangen?": ja — und das ist heute so gewollt.** Ob es so bleiben soll, ist eine Produktentscheidung und keine, die ein Härte-Track still trifft. Beide Fälle stehen als Test fest, damit eine spätere Änderung eine bewusste ist.

2 — P1: Der Inhaber konnte seinen eigenen Salon bewerten

`checkEligibility` hat den Salon nie geladen — und damit nie gefragt, wem er gehört. Der Weg ohne Buchungsbezug hat keine Vorbedingung außer „noch nicht bewertet“, also genügte:

```
POST /api/reviews
{ "salonId": "<eigener Salon>", "rating": 5, "comment": "Bester Salon!" }
```

Kundenbewertungen sind nicht double-blind, die Zeile entsteht also sofort mit `published: true`. Danach schreibt `updateSalonRating` sie nach `salons.avg_rating` und `salons.review_count` — und

genau diese beiden Werte stehen als `AggregateRating` im JSON-LD der Salonseite, auf den Kacheln der Startseite und in der Suche.

Die Gegenrichtung war längst zu: `/api/reviews/rental` weist `revieweeUserId === userId` ausdrücklich ab. Der ältere Kunden-Salon-Pfad hatte die Prüfung nie. Jetzt lädt `checkEligibility` den Salon und weist den Inhaber ab; nebenbei fällt damit auch eine Bewertung zu einer `salonId`, die es gar nicht gibt, weg.

3 — P1: Der Passwort-Zwang war eine Kette ohne Strom

Vollständig verdrahtet:

- `profiles.password_must_change` existiert in der Produktionsdatenbank.
- `decideAuthAccess` in der Middleware entscheidet mit `session.user.passwordMustChange` und leitet auf `/auth/change-password?forced=1` um bzw. beantwortet API-Aufrufe mit `password_change_required`.
- Die Seite dafür gibt es, mit eigenem Text für den `forced`-Fall.
- `/api/auth/change-password` löscht das Flag nach Erfolg wieder.

Gesetzt hat das Feld auf der Session **niemand**: `authorizeCredentials` hat die Spalte nicht einmal ausgewählt, und weder der `jwt` - noch der `session` -Callback haben sie je angefasst.

`!!session.passwordMustChange` war damit in jedem Request `false`. Ein Admin, der das Flag in der Datenbank setzt, hätte nichts bewirkt.

Jetzt: die Spalte wird beim Login gelesen und über beide Callbacks in die Session getragen. Bewusst beim Login und nicht pro Request: die Middleware läuft auf der Edge und liest ausschließlich den Token — eine DB-Abfrage steht ihr dort nicht zur Verfügung (deshalb prüft `getServerSession` Rolle und Sperre separat).

Daraus folgt der zweite Teil des Fixes: **`/auth/change-password` meldet nach Erfolg ab**. Ein Redirect auf die Zielseite liefere sofort wieder in denselben Zwang zurück — der Nutzer hätte sein Passwort geändert und säße in einer Schleife fest. Der neue Login stellt einen Token ohne das Flag aus; `/auth` zeigt dazu einen Hinweis, damit der Logout kein wortloser Rauswurf ist.

4 — P2: `POST /api/reviews/[id]/reply` machte aus allem eine 400

Die Autorisierung selbst war in Ordnung (sie steht in `replyToReview`, wo sie hingehört — die Action ist als Server Action ohnehin direkt aufrufbar). Die Route machte aber aus **jedem** Fehlschlag einen Eingabefehler: „nicht angemeldet“, „keine Berechtigung“ und „nicht gefunden“ waren für den Aufrufer nicht zu unterscheiden. Jetzt trägt die Action einen Status, und die Route reicht ihn durch. Der bisherige Test hatte die 400 für den fremden Aufrufer festgeschrieben; er erwartet jetzt 403.

Zwei kleinere Funde derselben Funktion mit erledigt: geantwortet werden konnte auf **jede** Zeile mit passender `salon_id`, also auch auf eine double-blinde Miet-Bewertung (die tragen aus Legacy-Gründen dieselbe `salon_id`) — die Antwort wäre nirgends sichtbar geworden, hätte aber `reply / replied_at` überschrieben. Und ein fehlgeschlagenes Update wurde verschluckt: der Anbieter las „gespeichert“, die Antwort stand nirgends.

Geprüft und ohne Befund

Der Vollständigkeit halber, weil „nichts gefunden“ nur dann etwas wert ist, wenn dasteht, wo gesucht wurde:

- **Provider-Isolation A ↔ B.** Produkte (`.eq('salon_id', ...)`), Leistungen (dito), Salon-Stammdaten (`getOwnedSalon`), Termine (Inhaber-Prüfung in `resolveBookingActor`), Compliance-Dokumente (Owner-Check), Uploads (Besitz), Miet-Umsätze (`salon_id in (eigene)`), Postfach. Alle prüfen Besitz, keine nimmt eine Fremd-ID an.
 - **Stripe Connect.** `provider_stripe_accounts` hängt am `user_id` der Session; `platform_transactions.provider_user_id` entsteht im Webhook aus dem DB-Join `rental_equipment → salons.owner_id`, nicht aus Metadaten des Requests. Der Payout-Cron wählt darüber aus. Kein Weg, die Auszahlung eines anderen Anbieters umzulenken.
 - **Bildupload in fremde Salons.** `/api/upload` prüft `salons.owner_id` gegen die Session, `/api/uploads` läuft über `requireOwnedSalon` / `ensurePrimaryListing`. Beide lösen den Zielsalon selbst auf, statt ihn aus dem Request zu nehmen.
 - **Fremde Bewertungen löschen.** Es gibt keine DELETE-Route auf `reviews` — weder für Anbieter noch für Kunden.
 - **Selbst-Verifizierung.** Weder `PATCH /api/provider/salon` noch `PATCH /api/me/salon` führen `is_verified` oder `is_active` in ihrem Schema; beide sind `.strict()`. Ein Anbieter kann sich nicht selbst freischalten oder entsperren.
-

Offen

- **is_verified auf der Geldstrecke** — siehe Befund 1. Produktentscheidung.
 - **getOrCreateSalonSeller** sucht die `sellers`-Zeile nur über `user_id`. Wer zwei Salons hat, bekommt für Salon B den Verkäufer-Datensatz von Salon A; das Produkt trägt dann `salon_id` von B und `seller_id` eines Verkäufers von A. Keine Kreuzung zwischen zwei *Anbietern*, aber innerhalb eines Kontos inkonsistent.
 - **Shop-Bestellungen** buchen weiterhin keine Provision (`platform_transactions`) und benachrichtigen den Verkäufer nicht — unverändert offen aus Track 14.
-

Tests

19 neue (1217 → 1236), alle 68 Dateien grün.

Datei	Inhalt
<code>src/__tests__/e2e/gesperrter-salon.test.ts</code>	10 Tests: Sperre auf Termin, Verfügbarkeit, Mietbuchung (inkl. „Stripe wurde nie gerufen“), Mietanfrage, Mietsuche, Inseratsdetail — plus die Gegenprobe, dass ein aktiver Salon weiter arbeitet, und die beiden bewusst nicht sperrenden Fälle
<code>src/__tests__/e2e/passwort-zwang.test.ts</code>	6 Tests: Profil → <code>authorizeCredentials</code> → JWT → Session → <code>decideAuthAccess</code> , in beide Richtungen
<code>src/__tests__/e2e/review-integrity.test.ts</code>	3 neue: Selbstbewertung abgewiesen (inkl. „ <code>avg_rating</code> hat sich nicht bewegt“), unbekannter Salon, fremde Kundin bewertet weiter